Document Number: <QMS Ref. No.>	Copy Number: <copy number>

TATA CONSULTANCY SERVICES

Trivandrum

PRE ILP – STRUTS2 LAUNCH PART- IV

Content Manual ***Version 1.2***

May 2014

DOCUMENT RELEASE NOTICE

Notice No.

Client

Project

Document details

Name	***Version No.***	***Author***	***Description***
PreILP_Struts2 Launch.doc	***1***	***Reshmi UR (311685)***	***Initial draft version of Struts 2 Launch content***

Revision details:

Action taken	***Preceding page No.***	***New page No.***	***Revision description***

Change Register serial numbers covered:

The documents or revised pages are subject to document control.

Please keep them up-to-date using the release notices from the distributor of the document.

These are confidential documents. Unauthorised access or copying is prohibited.

Approved by: _____ ***Date:*** _____

Authorised by: _____ ***Date:*** _____

DOCUMENT REVISION LIST

Client

Project

Document Name

Release Notice Reference (for release)

Rev. No.	***Revision date***	***Revision description***	***Page No.***	***Prev page***	***Action taken***	***Addenda/New page***	***Release Notice reference***

Contents

COURSE 4 STRUTS 2 – PART IV 5

4.1 Objective 5
4.2 Course Content 5
4.2.1 Course Objective 5
4.2.2 Struts 2 – Interceptors 5
4.2.2.1 Introduction 5
4.2.2.1.1 Interceptors - in general 5
4.2.2.1.1.1 init() 5
4.2.2.1.1.2 intercept() 6
4.2.2.1.1.3 destroy() 6
4.2.2.1.2 Interceptors - in Struts 6
4.2.2.2 Understanding Interceptors 6
4.2.2.2.1 Role of Interceptors in Struts2 Life Cycle 6
4.2.2.2.2 Configuring Interceptor in struts.xml 7
4.2.2.2.3 Stacking Interceptors 7
4.2.2.2.4 Specific Interceptors For An Action 9
4.2.2.2.5 Specific Interceptors For A Package 9
4.2.2.2.6 Custom Interceptors 10
4.2.2.3 Recap 11
4.2.2.4 Model Driven 11
4.2.2.4.1 Recap 16
4.2.2.5 i18n Interceptor 16
4.2.2.5.1 Understanding Resource Bundles 16
4.2.2.5.2 Resource Bundles in struts2.x 17
4.2.2.5.3 Resource Bundle for UI Tags 17
4.2.2.5.3.1 Changes in Struts.xml 17
4.2.2.5.3.2 Addition of .properties file 17
4.2.2.5.3.3 Calling key inside jsp 18
4.2.2.5.4 Resource Bundle for Errors and Messages 18
4.2.2.5.5 Resource Bundle for Action Class 19
4.2.2.5.6 Recap 19
4.2.2.6 Exception Mapping Interceptor 20
4.2.2.6.1 Exception Handling per Action 20
4.2.2.6.2 Global Exception Handling 20
4.2.2.6.3 Recap 21
4.2.2.7 Validation Interceptor 21
4.2.2.7.1 Recap 22
4.2.2.8 Benefits of Interceptors in Struts 2.X 22
4.2.3 Struts 2 – Aware Interfaces 22
4.2.3.1 Introduction 22
4.2.3.2 ApplicationAware Interface 23
4.2.3.2.1 ApplicationAware interface Example 23
4.2.3.2.1.1 Changes in Action class 23
4.2.3.2.1.2 Changes in redirect jsp 24
4.2.3.3 SessionAware Interface 24
4.2.3.3.1 SessionAware Interface Example 24
4.2.3.3.1.1 Changes in Action class 25
4.2.3.3.1.2 Changes in jsp 25
4.2.3.4 Servlet RequestAware Interface 26
4.2.3.4.1 ServletRequestAware Interface Example 26
4.2.3.4.1.1 Changes in Action class 26
4.2.3.5 Difference between Requestaware and ServletRequestAware Interface? 26
4.2.3.6 ServletResponseAware Interface 26
4.2.3.6.1 ServletResponseAware Interface Example 26
4.2.3.6.1.1 Changes in Action class 26

4.2.3.7 Parameter Aware Interface 27
4.2.3.7.1 Parameter Aware Interface Example 27
4.2.3.7.1.1 Changes in Action class 27
4.2.3.8 Recap 28
4.3 Summary 28

COURSE 4 STRUTS 2 – PART IV

4.1 Objective

Objective of this document is to provide a detailed understanding about Struts 2.x Interceptors and aware interfaces.

4.2 Course Content

4.2.1 Course Objective

At this point, you would have covered all the major aspects required for an application development using Struts 2.x

Here we will have a detailed discussion on the following topics.

- Role of interceptors in application development
- Major interceptors used commonly like, ModelDriven, i18N, exception handling interceptors.
- Role of aware interfaces in application development.
- Major aware interfaces commonly used for application development.

4.2.2 Struts 2 – Interceptors

4.2.2.1 Introduction

4.2.2.1.1 Interceptors - in general

An interceptor is a stateless class that follows the interceptor pattern, as found in Filter and in AOP languages.

Many Actions share common concerns. Some Actions need input validated. Other Actions may need a file upload to be pre-processed. Another Action might need protection from a double submit. Many Actions need drop-down lists and other controls pre-populated before the page displays.

Interceptors are objects that dynamically intercept Action invocations. They provide the developer with the opportunity to define code that can be executed before and/or after the execution of an action. They also have the ability to prevent an action from executing. Interceptors provide developers a way to encapsulate common functionality in a re-usable form that can be applied to one or more Actions.
Interceptors must be stateless and not assume that a new instance will be created for each request or Action. Interceptors may choose to either short-circuit the ActionInvocation execution and return a return code (such as Action.SUCCESS), or it may choose to do some processing before and/or after delegating the rest of the processing using ActionInvocation.invoke().

In the interceptor interface we have 3 methods

4.2.2.1.1.1 init()

void init()

Method init() is called after an interceptor is created, but before any requests are processed using intercept, giving the Interceptor a chance to initialize any needed resources.

4.2.2.1.1.2 intercept()

This method allows the Interceptor to do some processing on the request before and/or after the rest of the processing of the request by the ActionInvocation or to short-circuit the processing and just return a String return code.

String intercept (ActionInvocation invocation) throws Exception

Returns:

The return code, either returned from ActionInvocation.invoke(), or from the interceptor itself.

Throws:

Exception - any system-level error, as defined in Action.execute().

4.2.2.1.1.3 destroy()

void destroy()

Called to let an interceptor clean up any resources it has allocated.

4.2.2.1.2 Interceptors - in Struts

In Struts 2.x, we have Interceptors, if we want pre-processing and post-processing services for an action class. The tasks that are done by the Struts 2 framework, before and after an Action is executed are done by Struts 2 interceptors.
Interceptors are standard Java classes included in the Struts 2 core jar which are executed in a specific order.

An interceptor will intercept a client request, and execute some per-processing and post-processing services for Action class business logic. The main advantages of these interceptors are we can apply different-different pre-post services to the different-different actions, which means no need to apply same services to all Actions.

Struts2 comes with default list of Interceptors already configured in the application in struts-default.xml file. We can create our own custom Interceptors and plugin into a Struts2 based web application.

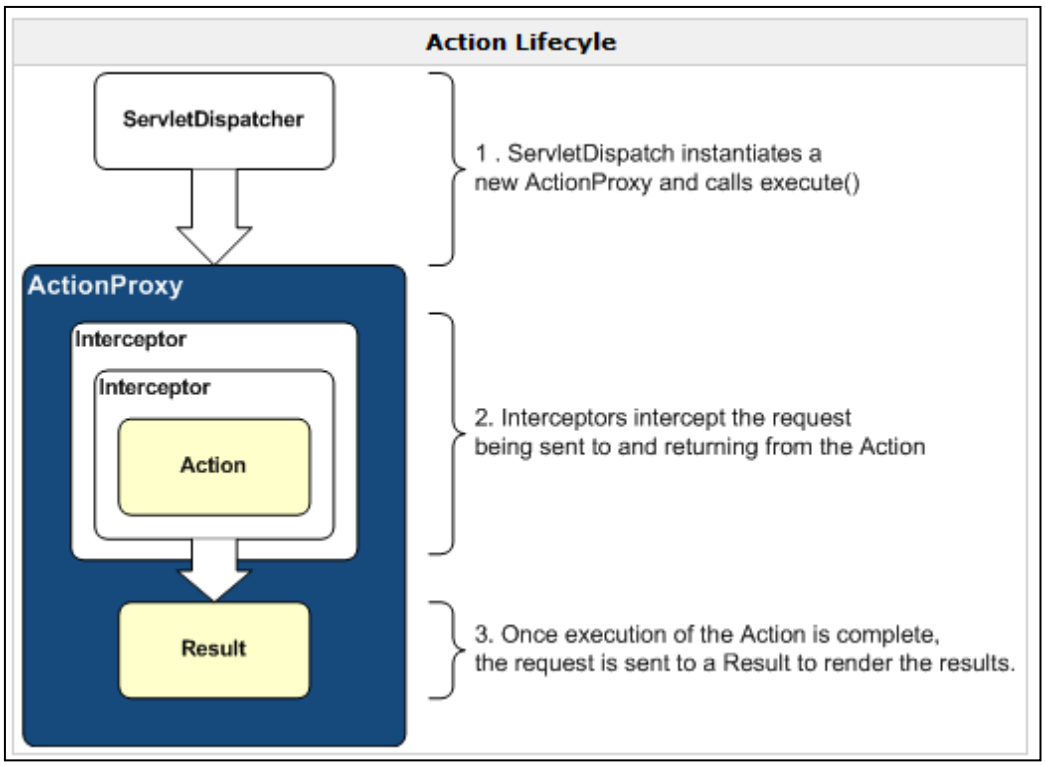
4.2.2.2 Understanding Interceptors

Interceptors "set the stage" for the Action classes, doing much of the "heavy lifting" before the Action executes.

All interceptors are classes, which implements interceptor interface, so we must override all methods in the interceptor interface.

4.2.2.2.1 Role of Interceptors in Struts2 Life Cycle

You have already seen the request processing life cycle of Struts2 in earlier sections.
Now we will see how an interceptor is involved in Struts2 Framework life cycle.



1. Request is generated by user and sent to Servlet container.
2. Servlet container invokes StrutsPrepareAndExecuteFilter which in turn determines appropriate action.
3. One by one Intercetors are applied before calling the Action. Interceptors perform tasks such as Logging, Validation, File Upload, Double-submit guard etc.
4. Action is executed and the Result is generated by Action.
5. Based on the result interceptor does some post-processing request.
6. The output of Action is rendered in the view (JSP, Velocity etc.) and the result is returned to the user.

4.2.2.2.2 Configuring Interceptor in struts.xml

Configure the interceptor in the struts.xml file as follows.

```
<!-- Default package Employee home -->
<package name="my-default" namespace="/" extends="struts-default">

  <interceptors>
     <interceptor name="timer" class=".."/>
     <interceptor name="logger" class=".."/>
  </interceptors>
```

4.2.2.2.3 Stacking Interceptors

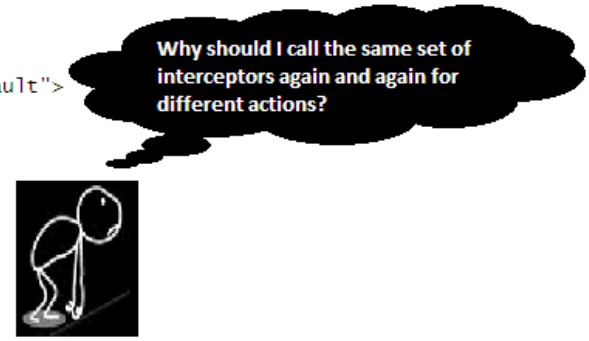
Mostly, same set of interceptors may be applicable for different action classes.
Following example shows scenarios where we may have to call same list of interceptors for multiple actions inside a package.

```
<package name="default" namespace="/" extends="struts-default">

    <action name="checkInAction"
        class=" CheckInAction" >
        <interceptor-ref name="timer"/>
        <interceptor-ref name="logger"/>
        <interceptor-ref name="defaultStack" />
        <result name="success">pages/checkIn.jsp</result>
    </action>

    <action name="checkOutAction"
        class=" CheckOutAction" >
        <interceptor-ref name="timer"/>
        <interceptor-ref name="logger"/>
        <interceptor-ref name="defaultStack" />
        <result name="success">pages/checkOut.jsp</result>
    </action>

</package>
```



Here both *checkInAction* and *checkOutAction* are calling the same list of interceptors.

To avoid this we have **interceptor stack** feature in Struts 2, which allows the developer to group a set of interceptors into a single unit called **stack name** and the action can invoke it via **stack name.**

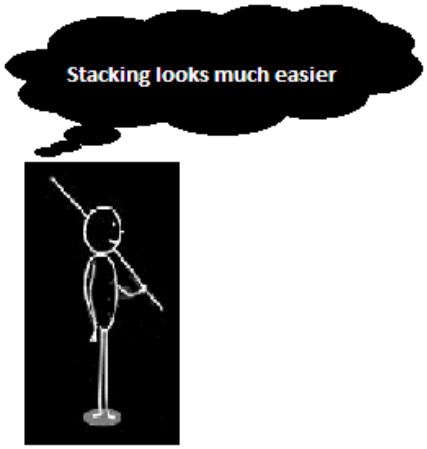
```
<package name="default" namespace="/" extends="struts-default">

<interceptors>
<interceptor-stack name="defaultStackWithLog">
        <interceptor-ref name="timer"/>
        <interceptor-ref name="logger"/>
        <interceptor-ref name="defaultStack" />
</interceptor-stack>
</interceptors>

<action name="checkInAction"
class=" CheckInAction" >
<interceptor-ref name="defaultStackWithLog"/>
<result name="success">pages/checkIn.jsp</result>
</action>

<action name="checkOutAction"
class=" CheckOutAction" >
<interceptor-ref name="defaultStackWithLog"/>
<result name="success">pages/checkOut.jsp</result>
</action>

</package>
```



Here *defaultStackWithLog* is an interceptor stack which includes *timer, logger and defaultStack* interceptors.

The default Interceptor stack is designed to serve the needs of most applications. Most applications will not need to add Interceptors or change the Interceptor stack.

<interceptor-ref name="defaultStack" />

This should be noted that interceptors are executed in the order, in which they have been configured.

4.2.2.2.4 Specific Interceptors For An Action

We can write specific interceptors for each action in the struts.xml as shown below.

```
<action name="register" class="com.tcs.ilp.register.action.Register" method="execute">
<interceptor-ref name="timer" />
<interceptor-ref name="logger" />
<interceptor-ref name="defaultStack">
    <param name="exception.logEnabled">true</param>
    <param name="exception.logLevel">ERROR</param>
</interceptor-ref>
<result name="success">thankyou.jsp</result>
<result name="input">register.jsp</result>
</action>
```

4.2.2.2.5 Specific Interceptors For A Package

Interceptors can also be written for a package as a whole. Those interceptors will be called for all actions written in the package.

```
<package name="basicstruts2" extends="struts-default" >
        <interceptors>
          <interceptor-stack name="appDefault">
             <interceptor-ref name="timer" />
             <interceptor-ref name="logger" />
             <interceptor-ref name="defaultStack" />
          </interceptor-stack>
         </interceptors>
         <default-interceptor-ref name="appDefault" />
        <!-- rest of package omitted -->
</package>
```

4.2.2.2.6 Custom Interceptors

Struts 2 provide provision to create Custom Interceptors if you would like to.
Let us see the steps to create our own interceptor.

Step 1: Create one java class which should implements from Interceptor interface (All interceptors in struts2 must implements from Interceptor interface only.)

*MyInterceptor.java

```java
package demo;

import com.opensymphony.xwork2.ActionInvocation;
import com.opensymphony.xwork2.interceptor.Interceptor;

public class MyInterceptor implements Interceptor
{

    @Override
    public void destroy() {
        // TODO Auto-generated method stub
        System.out.println("Interceptor destroy method is called....");
    }

    @Override
    public void init() {
        // TODO Auto-generated method stub
        System.out.println("Interceptor init method is called....");
    }

    @Override
    public String intercept(ActionInvocation invo) throws Exception {
        // TODO Auto-generated method stub
        String msg;
        System.out.println("Before Action invocation....");
        msg = invo.invoke();
        System.out.println("After Action invocation....");
        // Here String msg contains "success"
        return msg;
    }

}
```

Here, *MyInterceptor* is the interceptor created newly.

Override all abstract methods in *Interceptor* interface. Interceptor contains 3 abstract methods init(), destroy(), intercept(ActionInvocation inv)

Step 2: Finally configure our interceptor class in struts.xml file that's it, let us see the example

```
<?xml version="1.0" encoding="UTF-8"?>
    <struts>
    <include file="struts-default.xml"/>
    <package name="a" extends="struts-default">

    <interceptors>

                <interceptor name="myOwn" class="MyInterceptor"></interceptor>
                <interceptor-stack name="dummyName">
                      <interceptor-ref name="myOwn"/>
                </interceptor-stack>

    </interceptors>

        <action name="verify" class="LogingEx">
        <interceptor-ref name="dummyName"/>
        <result name="success">/success.jsp</result>
        </action>

    </package>
    </struts>
```

4.2.2.3 Recap

- Interceptor interface has 3 methods init(),intercept() and destroy
- The tasks that are done by the Struts 2 framework, before and after an Action is executed are done by Struts 2 interceptors.
- Interceptors perform tasks such as Logging, Validation, File Upload, Double-submit guard etc.
- Interceptors are configured in struts.xml
- Interceptor stack feature in Struts 2, allows the developer to group a set of interceptors into a single unit called stack name and the action can invoke it via stack name.
- We can either configure Interceptors for a package or for a specific action based on requirement.
- Custom Interceptors helps the developer to create user defined interceptors.

4.2.2.4 Model Driven

In Struts 2, action class itself is treated as model class. And Struts 2 request parameters are bound directly to fields in the actions class. Sometime the request fields keep in separate class (bean class) so it can be be used in other scenario also, and this domain class associate with Action class.

In order to implement the above concept, you create action class by implementing *com.opensymphony.xwork2.ModelDriven* interface. This interface contains ***getModel*** *()* method which return user defined domain object. Struts will then populate the fields of this object with the request parameters, and this object will be placed on top of the stack once the action is executed.

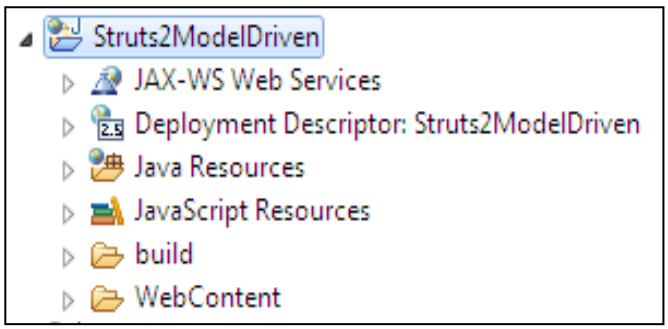
To implement ModelDriven action concept in your application, use ModelDriven interceptor, which is part of the default interceptor stack (defaultStack)so it is applied to all actions by default.

```
public class EmployeeAction extends ActionSupport implements ModelDriven<Employee>
{
}
```

Here *EmployeeAction* is our Action class name and *Employee* is Model class.

Let's demonstrate Model Driven action in Struts 2

Step 1: Create Dynamic web project: "Struts2ModelDriven"



Step 2: Copy necessary jar files in WEB-INF\lib folder

Step 3: Edit deployment descriptor web.xml

Steps 2 & 3 are explained in earlier sections.

Step 4: Create input page WebContent\jsp\index.jsp and View component WebContent\jsp\EmployeeSucess.jsp

index.jsp

```
<%@taglib prefix="s" uri="/struts-tags" %>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Model Driven Action Demo</title>
</head>

<body>
<h2>Employee Registration Form</h2>
<s:form action= "EmployeeAction1" method= "post" >
<s:textfield label= "Enter Code" name= "code" />
<s:textfield label= "Enter Name" name= "name" />
<s:textfield label= "Enter City" name= "city" />

<s:textfield label= "Enter Salary" name= "salary" value= "0" />
<s:submit label= "Submit" />
<s:reset label= "Reset" />
</s:form>
</body>
</html>
```

EmployeeSuccess.jsp

```
<%@taglib prefix="s" uri="/struts-tags" %>
<html>
<head>
<title>Employee Success Page</title>
</head>
<body>
<h2>Employee details are<br>
Employee Code :<s:property value= "code" /><br>
Employee Name :<s:property value= "name" /><br>
Employee City :<s:property value= "city" /><br>
Employee Salary :<s:property value= "salary" /><br>
<a href= "index.jsp" > Try Again</a>
</h2>
</body>
</html>
```

Step 5: Create Model class Employee .java

Employee.java

```
public class Employee {

    private String code;
    private String name;
    private String city;
    private Double salary;

    /*All getter & setter methods*/

    public String getCode()
    {
        return code;
    }
    public void setCode(String code)
    {
        this.code = code;
    }

    public String getName()
    {
        return name;
    }
```

Step 6: Create Action class EmployeeAction.java

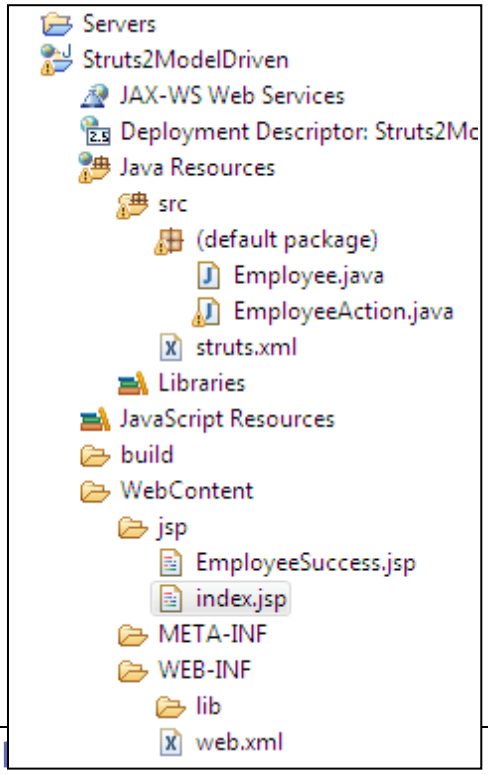
```
Employee.java    EmployeeAction.java
import com.opensymphony.xwork2.ActionSupport;
import com.opensymphony.xwork2.ModelDriven;

public class EmployeeAction extends ActionSupport implements ModelDriven<Employee>
{
    Employee emp = new Employee () ;
    @Override
    public Employee getModel() {
        // TODO Auto-generated method stub
        return emp;
    }
    @Override
    public String execute()
    {
        return SUCCESS;
    }
```

Step 7: Create src\struts.xml and add map configuration between View, Action and Results

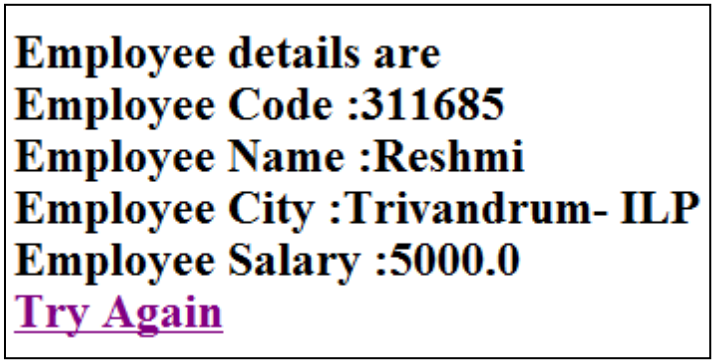
```
<!-- Default package which redirects to customer home -->
<package name="my-default" namespace="/" extends="struts-default">
 <action name= "EmployeeAction" class = "EmployeeAction" >
 <result name= "success" >/jsp/EmployeeSuccess.jsp</result>
 <result name= "input" >/index.jsp</result>
 </action>
</package>
```

Step 8: Verify the directory structure as below



Step 9: Run the application and get output

Fill the form & click submit button and see the output



4.2.2.4.1 Recap

- ModelDriven interface helps in associatind a bean class with an action class.
- ModelDriven interface contains ***getModel*** *()* method which return user defined domain object. ie, your bean class object.

4.2.2.5 i18n Interceptor

The word 'Internationalization' is often abbreviated to 'i18n'. This is widely used abbreviation, derived from the fact that there are 18 letters between the 'i' and the 'n'.

Here we will see the use of Struts 2 i18n interceptor usage as a resource bundle.
The same interceptor can be used as an internationalizing interceptor in case your project can support multiple languages.

Struts2 provides localization i.e. internationalization (i18n) support through resource bundles, interceptors and tag libraries in the following places:

- The UI Tags
- Messages and Errors.
- Within action classes.

4.2.2.5.1 Understanding Resource Bundles

Resource bundles are the file that contains the key/value pairs for the default language of your application.
If we want to get labels and error messages from an external file then we need to use resource bundle.

The simplest naming format for a resource file is:

bundlename_language_country.properties

Here bundlename could be ActionClass, Interface, SuperClass, Model, Package, Global resource properties.

Create the key message with the required value in the properties file.
All properties file should reside in the src folder

You can call the key value whenever needed in jsp or the action class

When you reference a message element by its key, Struts framework searches for a corresponding message bundle in the following order:

- **ActionClass.properties**
- **Interface.properties**
- **SuperClass.properties**
- **model.properties**
- **package.properties**
- **struts.properties**
- **global.properties**

4.2.2.5.2 Resource Bundles in struts2.x

The major configuration change required in order to use resource bundle in struts frame work is in struts.xml file.

<struts>
<constantname="struts.custom.i18n.resources" value ="form_labels" />

Here, *form_labels* is the name of property file which is used as resource bundle.
We can have multiple property files for an application. The name of property files should be declared above the package tags as shown above.

Following is the syntax for addition of values inside property file.

firstName=First Name
lastName=Last Name

Here, *firstName* is the reference variable used in your code (KEY) and *First Name* is the actual value displayed to the user (VALUE).

As explained above, we can use resource bundles in struts2 for UI tags, Messages & Errors and inside action class.
Now we will see some examples for each scenario.

4.2.2.5.3 Resource Bundle for UI Tags

It is best practice to have all the labels used in your application in the centralized location. We will see the changes required to include resource bundle for UI tags.

4.2.2.5.3.1 Changes in Struts.xml

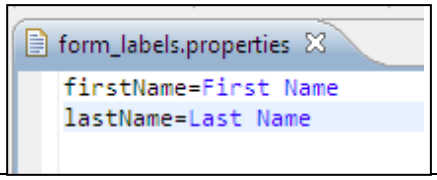
The changes in struts.xml to include property file is as highlighted.

```
<struts>
<constant name="struts.custom.i18n.resources" value="form_labels" />
<constant name="struts.devMode" value="true" />
<package name="uitagsdemo" extends="struts-default">
    <action name="PropertyDemo" class="Struts2HelloWorldAction" >
        <result name="success">Input.jsp</result>
    </action>
</package>
</struts>
```

form_labels is the name of property file we are using here.

4.2.2.5.3.2 Addition of .properties file

Your resource bundle should have extension as *.properties* and stored inside *src* folder.
Sample properties file is as follows.



4.2.2.5.3.3 Calling key inside jsp

```
<%@ page contentType="text/html; charset=UTF-8"%>
<%@ taglib prefix="s" uri="/struts-tags"%>
<html>
    <head>
    </head>
    <body>
        <h2> Struts 2 Property Labels Demo</h2>
        <s:form action="PropertyDemo">
            <s:textfield name="firstName" key="firstName" />
            <s:textfield name="lastName" key="lastName" />
            <s:submit label="Submit" />
        </s:form>
    </body>
</html>
```

Here *<s:textfield name="firstName" key="firstName" />* will search for value of key *firstName* inside *form_labels.properties*.
Your output jsp is as follows.

4.2.2.5.4 Resource Bundle for Errors and Messages

Resource bundle can be used as a standard storage for all error and success messages.
This will provide projects feasibility of storing all messages in a common location and developers can reuse from the standard message list rather than creating individual messages. This feature is very helpful particularly in case of larger projects.

You are already familiar with addActionError and *addFieldError* methods and their usage in action class.
When we use resource bundle for messages we will replace the actual message with key from resource bundle as shown below.

Without resource bundle

this.addActionError("Invalid User Name/Password");
this.addFieldError("password",”User name and password cannot be same");

With resource bundle

this.addActionError(getText("up.wrong"));
this.addFieldError("password",getText("pass.wrong"));

getText() is used to retrieve the value from a resource bundle and input for *getText()* is the key of the message to be retrieved.

Your resource bundle will look as below.

```
errorMessages.properties
up.wrong = Invalid User Name/Password
pass.wrong = User name and password cannot be same
```

4.2.2.5.5 Resource Bundle for Action Class

Just like we call resource bundles inside jsp, we can also call resource bundles inside our action classes.
To get the value stored in resource bundle, the method used is *getText(key inside resource bundle).*
This feature is useful, in case of errors/messages (as explained in the previous section), to validate values and to assign values to objects in an action class.

Example:

```
public String execute() throws Exception{
    String flag="failure";

    if(this.username.equals(getText("user.admin"))&& this.password.equals(getText("pas.admin"))){

        flag="success";
    }

    else{

        addActionError(getText("up.wrong"));
        }


    return flag;

}
```

```
errorMessages.properties
user.admin = TCS_Admin
pas.admin = TCS#2014
up.wrong = Invalid User Name/Password for Administrator.
```

Here, in execute () we are checking whether given credentials for Administrator matches with credentials provided by the user. The reference credentials are taken from *errorMessages.properties file.*

4.2.2.5.6 Recap

- Struts2 provides localization i.e. internationalization (i18n) support through resource bundles, interceptors and tag libraries.

- Resource bundles are the file that contains the key/value pairs for the default language of your application.
- Resource bundles needs to be configured in Struts.xml using *constant* tag.
- *'key'* is used to refer resource bundle values inside UI tags.
- *'getText()'* is used to refer resource bundle values inside action class.

4.2.2.6 Exception Mapping Interceptor

The Struts 2 framework provides the simple way to handle any uncaught exceptions generated by a web application and also provides robust exception handling, including the ability to automatically log any uncaught exceptions and redirect the user to a error web page. It works on exception interceptor which is part of default-stack in struts-default.xml file in any application.

There are two was to handle exception.

4.2.2.6.1 Exception Handling per Action

If you need to handle an exception in a specific way for a certain action you can use the exception-mapping node within the action node.
Let's demonstrate how to handle exception type in Struts 2.

```
<!-- Default package Employee home -->
<package name="my-default" namespace="/" extends="struts-default">

 <action name= "EmployeeAction" class = "EmployeeAction" >
 <exception-mapping result="error" exception="java.lang.NumberFormatException" />
 <result name= "success">/EmployeeSuccess.jsp</result>
 <result name= "input">/index.jsp</result>
 <result name="error">/EmployeeError.jsp</result>
 </action>

</package>
```

In the example we have discussed for Model Driven Interceptors, I have added a *NumberFormatException* while calling the action in struts.xml. So if *NumberFormatException* occurs inside the *execute()* method for *EmployeeAction* framework will search inside action tag for *EmployeeAction.* From the exception-mapping framework identifies the result name as *"error"* and *EmployeeError.jsp* will be displayed.

4.2.2.6.2 Global Exception Handling

If any exception has occurred in action and not configured the exception catching configuration with action configuration in struts.xml then framework searches in the global exception type and map with a result. This result type has to map using global result type.

```
<!-- Result pages for unmapped exceptions -->
<global-results>
<result name="checkexception">failureArithmetic.jsp</result>
<result name="null">failureNull.jsp</result>
<result name="excep">failureOthers.jsp</result>
</global-results>
<!-- List of unmapped exceptions  -->
<global-exception-mappings>
<exception-mapping result="checkexception" exception="java.lang.ArithmeticException" />
<exception-mapping result="null" exception="java.lang.NullPointerException" />
<exception-mapping result="excep" exception="java.lang.Exception" />
</global-exception-mappings>
```

In the above code, if NullPointer exception or Arithmetic exception is not mapped in the respective Action class then framework will search in the global exception. And if any other exception is not mapped in the Action class then framework will search in the global exception list. In all the cases respective result type is configured and framework will search the result name in the global results and forward to the target view page.
For example, if Nullpointer exception occurs, exception-mapping result name is *"null".* So the framework will search for result name *null* inside global-results and *failureNull.jsp* will be displayed.
It is recommended to make this interceptor as the first interceptor, so that it can handle all the exception whether it is thrown by other interceptors.

4.2.2.6.3 Recap

- Exception can be handled either globally or per action in Struts framework.
- 'exception-mapping' tag is used to map a specific exception to a result name.
- 'global-exception-mappings' tag is used to map all exceptions globally.

4.2.2.7 Validation Interceptor

This interceptor runs the action through the standard validation framework, which in turn checks the action against any validation rules (found in files such as *ActionClass-validation.xml*) and adds field-level and action-level error messages (provided that the action implements ValidationAware).

You are already familiar with usage of Validators in action class. In this section we will see, how exactly validators can be used in your action class.

Validation interfaces are defined inside struts-defaultxml.xml as follows.

```
<interceptor name="validation"
class="org.apache.struts2.interceptor.validation.AnnotationValidationInterceptor"/>
```

A default stack is also defined inside struts-default.xml which invokes the interceptor.

```
<!-- Sample validation and workflow stack -->
        <interceptor-stack name="validationWorkflowStack">
            <interceptor-ref name="basicStack"/>
            <interceptor-ref name="validation"/>
            <interceptor-ref name="workflow"/>
        </interceptor-stack>
```

This interceptor is often one of the last (or second to last) interceptors applied in a stack, as it assumes that all values have already been set on the action.

This interceptor does nothing if the name of the method being invoked is specified in the **excludeMethods** parameter. **excludeMethods** accepts a comma-delimited list of method names.

In the following case *myMethod* of the action class will not get validated.

```
<action name="someAction" class="com.examples.SomeAction">
   <interceptor-ref name="params"/>
   <interceptor-ref name="validation">
      <param name="excludeMethods">myMethod</param>
   </interceptor-ref>
   <interceptor-ref name="workflow"/>
   <result name="success">good_result.ftl</result>
</action>
```

In the following case only annotated methods of the action class will be validated.

```
<action name="someAction" class="com.examples.SomeAction">
   <interceptor-ref name="params"/>
   <interceptor-ref name="validation">
      <param name="validateAnnotatedMethodOnly">true</param>
   </interceptor-ref>
   <interceptor-ref name="workflow"/>
   <result name="success">good_result.ftl</result>
</action>
```

4.2.2.7.1 Recap

- Validation interceptor excludes the methods being invoked in the **excludeMethods** parameter. **excludeMethods** accepts a comma-delimited list of method names.

4.2.2.8 Benefits of Interceptors in Struts 2.X

To summarize we will see few benefits of using interceptors in struts.

1. Interceptor plays a crucial role in achieving high level of separation of concerns.
2. Struts2 interceptors are configurable; we can configure it for any action we want.
3. We can create our own custom interceptors to perform some common tasks such as request params logging, authentication etc. This helps us in taking care of common tasks at a single location, achieving low maintenance cost.
4. We can create interceptors stack to use with different actions.

4.2.3 Struts 2 – Aware Interfaces

4.2.3.1 Introduction

In struts2 we don't have any http specific objects by default just like in servlets. If at all we want any http related objects in our Action class then we need to implement our Action class from Aware Interfaces provided by struts framework.
We can say here frame work using one form of dependency injection of type interface for injecting required objects into an Action class of struts2.

Most commonly used Aware Interfaces of

- ApplicationAware Interface
- SessionAware Interface
- ServletRequestAware Interface
- ServletResponseAware Interface
- ParameterAware Interface

Every Aware interface provides a setter method, so we must override that setter method while implementing the particular aware interface, at run time struts 2 controllers will automatically calls that setter method and injects the required object into that Action class.

4.2.3.2 ApplicationAware Interface

We need to implement our Action class from ApplicationAware interface whenever our Action class needs to get context behavior, means we can share our data across all the files of the web application by putting in a global object.
When we implement our Action class from ApplicationAware interface then the controller doesn't inject exactly servlet context object, instead it will injects a map object and this will be created once by the controller and the same object will be injected to all files of the struts application.

4.2.3.2.1 ApplicationAware interface Example

Here we will see what all changes are required in the action class and corresponding jsp files when we want to map data using ApplicationAware interface.
The changes compared to normal are highlighted.

4.2.3.2.1.1 Changes in Action class

LogingEx.java

```
import com.opensymphony.xwork2.ActionSupport;
import org.apache.struts2.interceptor.*;
import java.util.Map;
public class LogingEx extends ActionSupport implements ApplicationAware{
    private String uname,country;
    private int age;
    Map m;
    public String getUname() {
        return uname;    }
    public void setUname(String uname) {
        this.uname = uname;
    }
    public String getCountry() {
        return country;
    }
    public void setCountry(String country) {
        this.country = country;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void setApplication(Map m)
    {
        this.m=m;
    }
    public String execute()
    {
        m.put("a",uname);
        m.put("b", age);
        m.put("c",country);
        return SUCCESS;    }}
```

1 2 3 4 5

4.2.3.2.1.2 Changes in redirect jsp

```
<%@ taglib prefix="s" uri="/struts-tags" %>

Hello <s:property value="#application.a" />
Your age is:  <s:property value="#application.b" />
country:  <s:property value="#application.c" />
```

Here a,b and c are the mapping given for uname, age and country respectively.

4.2.3.3 SessionAware Interface

We need to implement our Action class from SessionAware interface in order to get HTTP Session behavior into our Action class.

If we implement from SessionAware interface we need to override the method setSession() by SessionAware in our action class. If we implement our action class from SessionAware interface then struts 2 controller doesn't inject exactly session object, but it will injects a Map object with similar behavior.

For each Action class or a jsp visited by the same client, the controller injects the same map object, the controller creates a new map object for each client, it means one map object per session (browser).

ie,

No. of clients = No. of map objects created by controller

4.2.3.3.1 SessionAware Interface Example

Here we will see what all changes are required in the action class and corresponding jsp files when we want to map data using SessionAware interface.
The changes compared to normal are highlighted.

4.2.3.3.1.1 Changes in Action class

```
LogingEx.java
import com.opensymphony.xwork2.ActionSupport;
import org.apache.struts2.interceptor.SessionAware;
import java.util.Map;
public class LogingEx extends ActionSupport implements SessionAware{
    private String uname,country;
    private int age;
    Map m;
    public String getUname() {
        return uname;    }
    public void setUname(String uname) {
        this.uname = uname;
    }
    public String getCountry() {
        return country;
    }
    public void setCountry(String country) {
        this.country = country;
    }
    public int getAge() {
        return age;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void setSession(Map m)
    {
        this.m=m;
    }
    public String execute()
    {
        m.put("a",uname);
        m.put("b", age);
        m.put("c",country);
        return SUCCESS;    }}
```

1 2 3 4 5

4.2.3.3.1.2 Changes in jsp

```
<%@ taglib prefix="s" uri="/struts-tags" %>

<s:form action="next">
Name:<s:property value="#session.a" /><br>
Age:<s:property value="#session.b" />
<s:submit value="next" />
</s:form>
```

Here a and b are the mapping given for stuname and stuage respectively.
We can also use SessionMap instead of Map.

4.2.3.4 Servlet RequestAware Interface

We need to implement our Action class from ServletrequestAware interface in order to get access to HTTPServletrequest object.

4.2.3.4.1 ServletRequestAware Interface Example

4.2.3.4.1.1 Changes in Action class

```
import javax.servlet.http.HttpServletRequest;
import org.apache.struts2.interceptor.ServletRequestAware;

public class LocaleAction implements ServletRequestAware{

        HttpServletRequest request;

        //business logic
        public String execute() {
                String param = getServletRequest().getParameter("param");
                return "SUCCESS";

        }

        public void setServletRequest(HttpServletRequest request) {
                this.request = request;
        }

        public HttpServletRequest getServletRequest() {
                return this.request;
        }
}
```

4.2.3.5 Difference between Requestaware and ServletRequestAware Interface?

RequestAware and ServletRequestAware both makes your action deal with the servlet request, but in different ways. RequestAware gives you the attributes in the servlet request as a map (key as attribute name and value is the object added).But ServletRequestAware gives you the HttpServletRequest object itself giving you more flexibility.

4.2.3.6 ServletResponseAware Interface

Action class doesn't have access to ServletResponse object by default, but when working in Servlet environment, ServletResponse is needed in Action class and that is injected in Action class by the using ServletResponseAware interface. It has one method setServletResponse(HttpServletResponse hRes) which sets ServletResponse object in Action class.

4.2.3.6.1 ServletResponseAware Interface Example

4.2.3.6.1.1 Changes in Action class

```
import com.opensymphony.xwork2.ActionSupport;
import org.apache.struts2.interceptor.ServletResponseAware;
```

```
public class ServletResponseAwareAction extends ActionSupport implements
ServletResponseAware {
   HttpServletResponse httpResponse ;
   public void setServletResponse(HttpServletResponse hRes) {
        this.httpResponse = hRes ;
   }
   public String execute() throws Exception {
     httpResponse.setAttribute("user","Niraj") ;
     HttpSession session = httpResponse.GetSession();
     return SUCCESS ;
   }
}
```

4.2.3.7 Parameter Aware Interface

The ParameterAware interface is used when Action wants to handle input parameters. All the input parameters with name-value are set in parameter Map. When Action requires HTTP request parameters, this interface must be implemented in Action class. Another use of this interface is to propagate parameters to internally instantiated data object. All the parameter values for a given name returns String, hence Map is of type String[] . ParameterAware interface has one method named setParameter (Map map) which sets the Map of input parameters in Action class. Action class uses this map object to access the input parameters or propagate them to other objects.

4.2.3.7.1 Parameter Aware Interface Example

4.2.3.7.1.1 Changes in Action class

```
ParameterAwareAction.java
package com.test.customer.action;

import com.opensymphony.xwork2.ActionSupport;
import org.apache.struts2.interceptor.ParameterAware;
import java.util.Map;

public class ParameterAwareAction extends ActionSupport implements ParameterAware {

    /**
     *
     */
    private static final long serialVersionUID = 1L;
    Map paramMap ;
    public void setParameter(Map param) {
          this.paramMap = param ;
    }
    public String execute() throws Exception {
       paramMap.put("user","Reshmi") ;
       return SUCCESS ;
    }
    public void setParameters(Map<String, String[]> arg0) {
        // TODO Auto-generated method stub

    }
}
```

4.2.3.8 Recap

- In order to invoke http specific objects in our Action class we need to implement our Action class from Aware Interfaces provided by struts framework.
- We need to implement our Action class from ApplicationAware interface whenever our Action class needs to get context behavior, means we can share our data across all the files of the web application by putting in a global object.
- We need to implement our Action class from SessionAware interface in order to get HTTP Session behavior into our Action class.
- We need to implement our Action class from ServletrequestAware interface in order to get access to HTTPServletrequest object.
- ServletResponse is injected in Action class by the using ServletResponseAware interface.
- ParameterAware interface is implemented in Action class, when Action requires HTTP request parameters.

4.3 Summary

The sections covered here are capable to enable the developers in developing dynamic projects using Struts framework. There are many additional interfaces and interceptors provided by Struts framework. The usage of those can be decided by the developer based on specific application requirements.
Few of those will be covered as part of future documents.